

Chameleon-SAT: An Adaptive Boolean Satisfiability Accelerator Using Mixed-Signal In-Memory Computing for Versatile SAT Problems

Iris Ying Chou, Hao Kong, Yi Huang, Jianfeng Zhu, Wenping Zhu, Shaojun Wei, Aoyang Zhang, and Leibo Liu
Tsinghua University, Beijing, China

Abstract—Boolean satisfiability (SAT), the first proven non-deterministic polynomial-complete problem, is crucial in data-intensive applications. Different applications have a wide spectrum of SAT problem sets (scale, complexity) and also various solution requirements (algorithm completeness, speed). Current SAT solvers are insufficient for providing ideal solutions under different scenarios. This work presents the Chameleon-SAT, the first ASIC-based SAT accelerator that can support local search, Davis-Putnam-Logemann-Lovel, Conflict-Driven Clause Learning algorithms, while leveraging the efficient mixed-signal in-memory computing architecture to achieve orders-of-magnitude improvements in speed compared to the prior SAT solvers. By judiciously selecting the reconfiguration mode, Chameleon-SAT is able to solve a wide range of the SAT problems to achieve small-scale, high-complexity cases ($\geq 90\times$ for 20 variables/ 86 clauses, satisfiable problems), medium-scale, structured cases ($\geq 19\times$ for 50 variables/ 215 clauses, unsatisfiable problems), and large-scale, high-complexity cases ($\geq 7\times$ for 100 variables/ 430 clauses, satisfiable problems).

Index Terms—Boolean satisfiability, SAT solver, mixed-signal, processing-in-memory, accelerator, algorithm portfolio, adaptive accelerator.

I. INTRODUCTION

Boolean satisfiability (SAT) is the problem of determining if an interpretation exists that satisfies a given Boolean formula. It plays a crucial role in computer science [1], [2], with far-reaching implications for various fields, such as logic circuit design [3], software verification, database management, artificial intelligence [4], bioinformatics [3], and cryptography [2]. The importance of this problem lies in the fact that it is directly related to the solvability and efficiency of solving complex problems in these fields [5].

There are two main types of algorithms for dealing with SAT problems [6]: incomplete algorithms, such as stochastic local search, and complete algorithms, such as Davis-Putnam-Logemann-Lovel (DPLL) [7], [8] and conflict-driven clause learning (CDCL) [9], [10]. Incomplete algorithms, such as stochastic algorithms [11], are known for their efficiency and flexibility. They are usually based on search and optimization strategies to quickly find possible solutions in a large solution space. Complete algorithms, such as DPLL and CDCL, focus more on the rigor and the precision of logical derivation. In the data-intensive applications, demand has

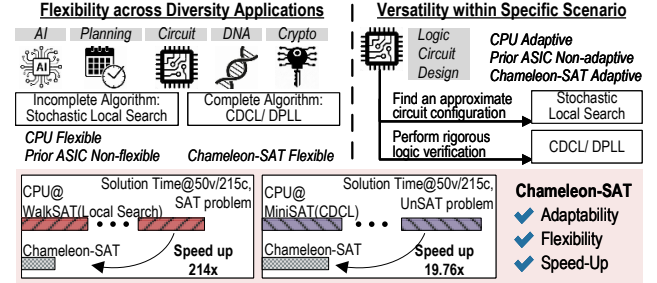


Fig. 1. Illustration of Chameleon-SAT solver's adaptability, flexibility, and speed across diverse applications, comparing CPU solution time and speedup for SAT/UnSAT problems using local search and CDCL methods.

surged for processing SAT problems, challenging algorithm speed and flexibility. The vast data volume, increased complexity, and diverse applications demand innovative computational methods and architectures.

First, SAT problems are widely applicable across diverse domains, and spanning these domains entails rapidly adjusting and optimizing algorithm execution to fit specific application scenarios and requirements. Supporting multiple algorithms and selecting the most appropriate ones for acceleration based on different contexts have become key to meeting this demand. Second, solving SAT problems requires different stochastic versus complete algorithms, even within specific identical scenarios. This is because different SAT algorithms exhibit very different solution times and hardware energy consumption. For example, in logic circuit design, local search-based algorithms can be employed to rapidly identify an approximate circuit configuration. To ensure that the circuit is correct and optimal, we then need to perform rigorous logic verification and optimization using deductive algorithms to ensure that the circuit meets all design specifications and performance requirements. However, current ASIC-based SAT accelerators support only one algorithm, making it hard to adapt to different requirements.

In response to these needs (as shown in Fig. 1), Chameleon-SAT is introduced. This mixed-signal in-memory computing accelerator not only supports externally designated algorithms but also features an adaptivity switch and internally selects the optimal algorithm for acceleration based on the characteristics and solution stages of the actual problem. The key contributions are summarized as follows.

Corresponding author: Aoyang Zhang (aoyang@tsinghua.edu.cn), and Leibo Liu (liulb@tsinghua.edu.cn).

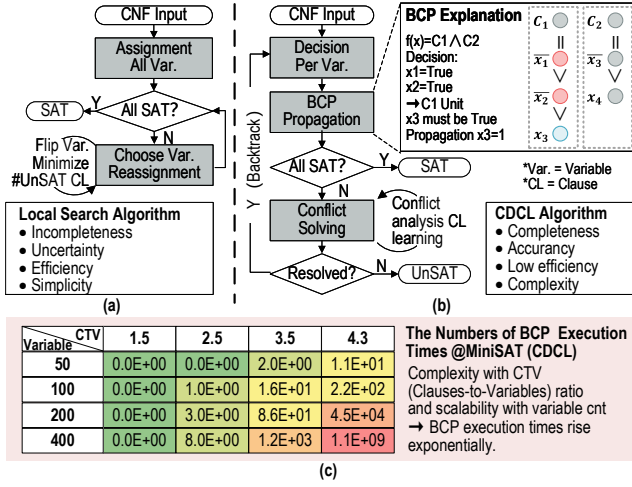


Fig. 2. (a) Incomplete algorithm (Local Search): Initializes, iteratively refines via heuristics, and verifies until solved or timed out. (b) Complete algorithm (CDCL): Initializes, constructs via decisions and BCP, backtracks, learns, and repeats until solved or proven unsolvable. (c) The numbers of BCP execution times across varying complexity and scalability.

- We designed Chameleon-SAT, the first SAT accelerator that supports both complete and incomplete SAT-solving algorithms (local search, CDCL, DPLL). By comparing previous ASICs that support only one algorithm (either complete or incomplete), Chameleon-SAT can adapt to various configurations to fit the requirements of different applications.
- We built an adaptive selection scheme that judiciously selects the optimal algorithm based on the defined quality function, thereby addressing the large variations (different CTV and variables) in a wide range of SAT problems.
- Chameleon-SAT utilizes a mixed-signal in-memory computing (IMC) architecture to enhance the level of speed in the hardware. Moreover, the data energy consumption is minimized thanks to the IMC architecture.
- Chameleon-SAT achieved small-scale, high-complexity cases ($\geq 90\times$ for 20 variables/ 86 clauses, satisfiable problems), medium-scale, structured cases ($\geq 19\times$ for 50 variables/ 215 clauses, unsatisfiable problems), and large-scale, high-complexity cases ($\geq 7\times$ for 100 variables/ 430 clauses, satisfiable problems).

II. PRELIMINARY

A. Boolean SAT Problems

The SAT problem, focuses on determining whether a given Boolean expression is true. The problem can also be framed as a satisfiability challenge for propositional logic formulas and is commonly represented as a conjunctive normal form (CNF). In CNF, clauses are composed of **OR**-connected literals, and the entire formula is connected by **AND**. Literals can be positive or negative, representing the variable or its negation.

For example, consider the following CNF formula:

$$f = (x1 \vee \neg x2) \wedge (x2 \vee x3) \wedge (\neg x1 \vee \neg x3).$$

Here, we have three variables, $x1$, $x2$, and $x3$, and three clauses. Each clause is a disjunction (**OR**) of literals, and the entire formula is a conjunction (**AND**) of these clauses. The task is to find an assignment of 0 or 1 to each variable such that all the clauses evaluate to true simultaneously.

Depending on the number of literals in each clause, SAT problems are further categorized into k -SAT problems, such as 2-SAT or 3-SAT. In terms of computational complexity, SAT problems are proven to be nondeterministic polynomial complete, with 3-SAT serving as a typical example. Notably, in k -SAT problems, as the ratio of clauses to variables (CTV) changes, the problem's satisfiability undergoes a dramatic transition from being always satisfiable to being always unsatisfiable—a phenomenon known as the SAT phase transition. Specifically, in 3-SAT problems, a CTV of 4.3 is recognized as the critical threshold marking the transition from satisfiability to unsatisfiability, and problems at this threshold are defined as hard SAT problems.

B. SAT Algorithms

The algorithms for solving the Boolean satisfiability problem are classified into two main categories: complete search algorithms and incomplete stochastic local search algorithms.

Complete search algorithms, exemplified by DPLL and CDCL, constitute the classical approach. DPLL, known for its guarantee of completeness and correctness, offers a medium level of efficiency. It is particularly suited for small to medium-sized problems with moderate complexity, striking a balance between solution quality and processing speed. On the other hand, CDCL (illustrated in Fig. 2b), while incurring additional overhead due to clause learning, excels at solving large-scale, high-complexity problems with strict correctness requirements. Both DPLL and CDCL employ bifurcated branching search and integrate various optimization techniques, such as preprocessing, look-ahead, backtracking, nontemporal backjumping, and conflict-driven clause learning, to enhance search efficiency. Boolean constraint propagation (BCP), a core step in these algorithms, becomes exponentially more complex and scalable with increasing problem size (illustrated in Fig. 2c), thereby significantly impacting the solving process's efficiency.

In contrast, stochastic local search algorithms (shown in Fig. 2a), including GSAT, WalkSAT, and simulated annealing, focus on satisfying more clauses by continuously modifying assignments. Although they cannot prove unsatisfiability, they often swiftly find acceptable solutions. These algorithms are highly efficient and capable of rapidly processing large datasets, making them suitable for scenarios in which speed is crucial and a certain error rate is tolerable. Within this category, the efficiency of clause computation during each assignment modification is pivotal in reducing the overall solving time.

C. SAT Solvers Insufficiency

In the context of the significant data era, SAT solvers face unprecedented challenges. With the increasing size, complexity, and diversity of SAT problems, solvers need more vital

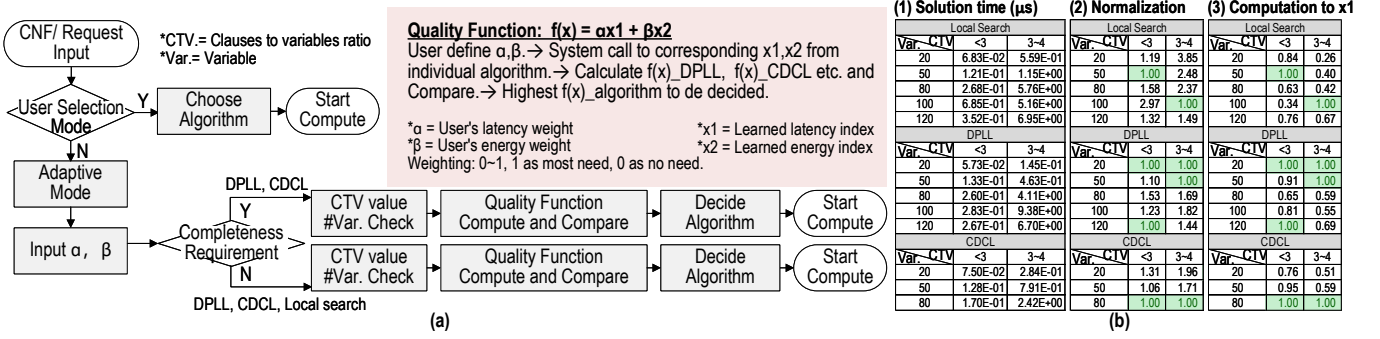


Fig. 3. Framework of Chameleon-SAT. (a) Flow of Chameleon-SAT operation. After inputting the SAT problem in CNF, the user can self-select an algorithm from a list to execution or adapt by specifying weights for solution time α , energy β , and completeness needs. Chameleon-SAT then chooses the optimal algorithm based on problem features (e.g., CTV and variable count) and a quality function score. (b) Depiction of the derivation of x_1 in the quality function from test sets generated using conditional matrix analysis. This involves normalizing solution times and computing their reciprocals to obtain x_1 .

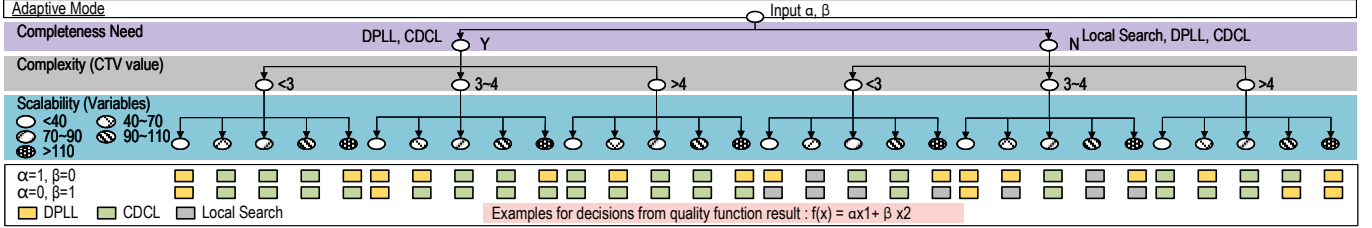


Fig. 4. Illustration of the branching process and examples of parameter settings in adaptive mode. After inputting the weights α (for solution time) and β (for energy), along with the completeness requirement, the system extracts CTV and the number of variables, categorizing the problems into 30 groups. Examples are provided for cases where $\alpha = 1, \beta = 0$, and $\alpha = 0, \beta = 1$, showcasing the most suitable algorithm with the highest quality function score for each group.

adaptability, more efficient algorithmic combining ability, and lower data processing overhead. However, current solvers still have significant deficiencies in these areas. To meet these needs, researchers have developed a variety of hardware [12] and software [13], [14] acceleration schemes. On the software side, solvers such as MiniSAT [15] are the mainstream in the industry, and Kissat [16] improves the solving capability by improving the data structure and algorithm optimization. ParaFROST [17] combines the CPU and GPU's computational power, significantly improving the performance of SAT solving through parallel processing. However, as the problem size increases, the data processing overhead becomes a pressing issue.

Hardware accelerators have emerged to overcome this limitation. Among them, FPGA-based solvers [18] have been implemented to also accelerate SAT-solving algorithms through customized hardware design. ASIC solutions, such as BRIM [19], Snap-SAT [20], and VIP-SAT [21], further improve the efficiency of solving problems by enhancing hardware functionality. Despite the performance improvement of these hardware accelerators, these architectures accelerate only one particular algorithm. In summary, recent SAT solvers have made some progress in hardware and software acceleration. However, they cannot support multiple SAT algorithms to meet the requirements of solvers in modern data-intensive applications.

III. CHAMELEON-SAT FRAMEWORK

A. Overview and Operation Flow

Chameleon-SAT supports the user selection mode and the adaptive mode, as shown in Fig. 3a. First, the user must enter

the CNF form of the SAT problem. Then, the system will ask the user if they want to use the self-selected algorithm mode.

(1) Self-selected algorithm mode: If the user selects yes, the system will provide a list of algorithms from which the specified algorithm can be selected. Once selected, the hardware will execute the appropriate algorithmic operations as instructed.

(2) Adaptive mode: Chameleon-SAT will enter the adaptive mode if the user chooses not to use the self-selected algorithm mode. In the adaptive mode, the user first needs to input the coefficient values of α and β , where α represents the weight of the solution time and α represents the weight of the energy. Immediately after that, the user needs to choose whether the solution's completeness is required. If completeness is required, Chameleon-SAT will choose the most suitable completeness algorithm (e.g., DPLL or CDCL). If a completeness requirement is unnecessary, the system will choose from algorithms including local search, DPLL, and CDCL. Next, the system extracts features of the problem, such as CTV and the number of variables. To determine the most suitable algorithm for a particular SAT problem, a quality function is defined. It considers both solution time and energy to decide the Chameleon-SAT's operation under different scenarios.

The quality function $f(x)$ is the key to evaluating the merits of an algorithm:

$$f(x) = \alpha x_1 + \beta x_2.$$

Here, the α and β in the function are weighting coefficients given by the user, while x_1 (solution time index) and x_2 (energy index) are built from a library of already learned

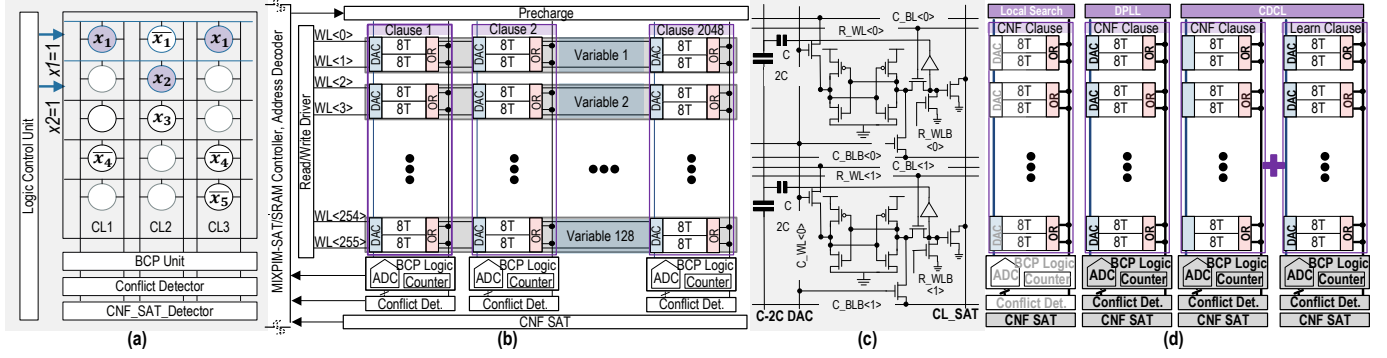


Fig. 5. (a) Chameleon-SAT architecture: Maps SAT to in-memory computing for parallel clause computation and BCP via WL activation. (b) SRAM array top-view: Columns = clauses, pairs = variables, literals stored, even/odd WLs assign 1/0. (c) SRAM cell pair: Represents variable, C-2C for DAC, CL-SAT for **OR** in clauses, illustrates DPLL/CDCL BCP. (d) PIM column usage in algorithms: Local search (**OR** with CL-SAT), DPLL (BCP, conflict detection), CDCL (learning clauses for pruning).

data corresponding to the values of the variables for different algorithms at different scales (i.e., the number of variables) and at different complexities (e.g., CTV). The values of α and β are limited to a range between 0 and 1, and the sum must equal 1. When a weight is given to 0, it indicates that there is no demand for that index; when it is given to 1, it indicates that there is a strong demand for that index. For example, if α is given to 1 and β is given to 0, it means that the optimization goal is on solution time instead of energy, Chameleon-SAT will take only the solution time to be the criteria to determine the most suitable algorithm. By selecting different coefficients of α and β , the quality function can take into account both energy and the solution times of different SAT algorithms under the Chameleon-SAT solver. Moreover, even for the same SAT problem (same CTV and variables), the user may have different requirements of energy and latency. For example, on the server side, the solution time can be the priority to optimize, while on the terminal side, the energy consumption is more important. Depending on the user demand, Chameleon-SAT can determine the coefficient value to find the most suitable solution for a specific problem.

Fig. 3b shows how the x_1 and x_2 variables are obtained in the quality function. These variables are based on the results of a group of test sets, which are derived through conditional matrix analysis and the corresponding operations. To be specific, we first normalize the solution times of different algorithms under the same condition (same CTV and variables). After normalization, we calculate the reciprocal of each result to obtain the value of the x_1 variable. For example, In Fig. 3b, when the variables are equal to 50 and the CTV is less than 3, local search, DPLL, and CDCL algorithms have solution times of 0.121 μ s, 0.133 μ s, and 0.128 μ s, respectively. The normalization process takes the minimum solution time, 0.121 μ s, to be the denominator. It normalizes the solution times to be 1, 1.1, and 1.06, respectively. The subsequent reciprocal process then converts the normalized number to between 0 and 1 for later quality function score calculation. Note that, the energy variable x_2 uses the same methodology to calculate the corresponding value of each algorithm under the same condition. Finally, the system compares the quality function scores of the different algorithms, selects the algorithm with

the highest score, and initiates the hardware to execute the corresponding algorithm for solving.

Fig. 4 details the exact process of parameter branching in adaptive mode and provides examples. In the adaptive mode, by inputting α and β weights and incorporating completeness requirements, Chameleon-SAT is able to determine the optimal solution under a given CTV as well as the number of variables. Currently, these variables are categorized into 30 different clusters. To illustrate the process more concretely, we provide two examples.

When α is equal to 1 and β is equal to 0, the solution time is mainly taken into account. This scenario is prepared for the server side, where the solution time can be the key parameter. Chameleon-SAT accelerator can determine the corresponding algorithms that need to be selected under different cases (different CTV and variables). Similarly, when α is equal to 0 and β is equal to 1, the energy consumption then becomes important. This scenario happens on the terminal side, where the energy consumption can be the important parameter that needs to be optimized. Within these different clusters, we look for the algorithms with the highest scores in the quality function that are considered the most suitable for their respective clusters.

B. Hardware Architecture

To accelerate SAT problem solving while maintaining low energy consumption in hardware design, we conducted a detailed analysis of the key steps and implemented targeted optimizations. Our focus was on two critical areas: the frequent clause matrix satisfiability operations following variable assignment flips in local search algorithms and the performance bottlenecks in the BCP steps of the CDCL and DPLL algorithms. We adopted IMC [22], [23], [24], [25], [26], [27] technology to address these core operations. IMC significantly reduces data handling overhead between the processor and memory by performing computations directly within the storage units. This leverages the parallel processing capabilities of the storage array, resulting in faster algorithm execution and lower energy consumption.

Fig. 5a illustrates how SAT problems are mapped and operated within the Chameleon-SAT IMC framework. By

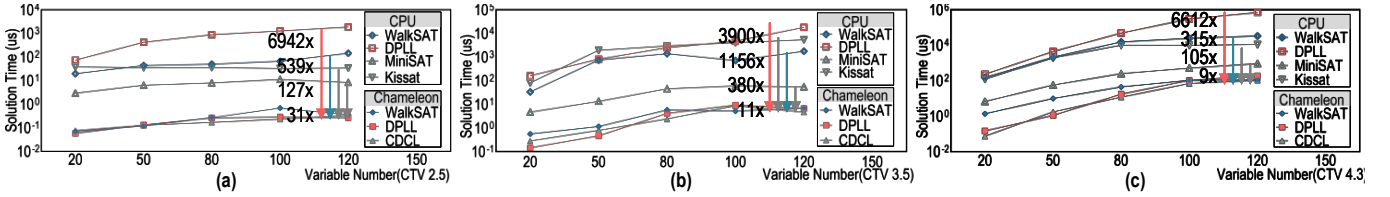


Fig. 6. Chameleon-SAT hardware accelerator outperforms CPU solvers in SAT problems.

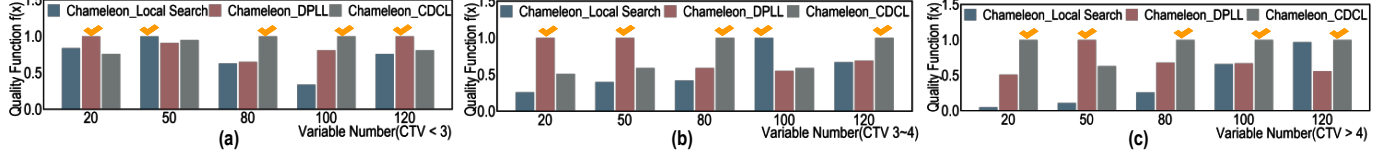


Fig. 7. Presentation of optimal SAT algorithms through quality function by scalability (variables) and complexity (CTV) with user assignment $\alpha = 1$, $\beta = 0$. Generally, DPLL suits small sizes (≤ 50 variables); for large (≥ 80 variables) and high complexity problems, CDCL and local search are best, with CDCL excelling in large-scale problems. CDCL for completeness; local search without.

activating the corresponding word lines (WLs) for variable assignments, we enable parallel computation of clauses in local search and CDCL/DPLL scenarios while supporting the BCP coding process. The top-view architecture (Fig. 5b) reveals the SRAM array matrix, in which each column represents a logical clause, and each pair corresponds to a logical variable. Literals (variables with signs) are stored within the clauses. WL activation is used to assign values to variables: even WLs represent variables assigned as 1, while odd WLs represent variables assigned as 0. Its mapping supports parallel internal clause computing.

Furthermore, we incorporated a mixed-signal design to tackle the complexity of variable identification coding in the BCP step. This design combines the precise control of digital circuits with the efficient processing power of analog circuits, thereby enhancing the variable identification coding process in BCP. Fig. 5c shows a schematic representation of an SRAM cell pair that serves as a variable with C-2C for digital-to-analog conversion (DAC) and CL-SAT for **OR** computation within the clause. Chameleon-SAT leverages the circuit design for the local search, DPLL, and CDCL algorithms to address computational bottlenecks, as illustrated in Fig. 5d.

In local search algorithm implementation, it several iterative flips are required to determine the satisfiability of CL-SAT. The proposed IMC architecture eliminates the read/write and movement of data between memory and the processor, enabling parallel operation of all clauses. Note that because there is no BCP operation in the local search algorithm, BCP logic and the conflict detection block are not activated.

In DPLL algorithm implementation, in addition to applying CL-SAT, the DPLL algorithm combines BCP logic and conflict detection functions for efficient propagation and conflict resolution. Thanks to the C-2C DAC, the BCP unit can operate in parallel by reading out the DAC output voltage, further enhancing the algorithm's performance.

Similar to DPLL, the CDCL algorithm further refines the solution process based on DPLL. It generates new learning clauses through learning and writes these clauses into SRAM to help constrain the search space in the next iteration to prune

the solution space more efficiently.

IV. EVALUATION

A. Experiments Setup

In the evaluation, Cadence Virtuoso was employed for SPICE simulations to assess the latency, power, and area of the in-memory computing unit. The digital components were designed in SystemVerilog and synthesized using Synopsys tools. All analog and digital designs and simulations were based on TSMC 28 nm technology. The in-memory computing unit was RTL abstracted for performance verification, ensuring accurate testing and reliability. We used Synopsys VCS and Verdi for simulation and debugging and benchmarked our results against CPU-based SAT solvers on an AMD Ryzen 9 9950X workstation.

B. Results and Analysis

1) *Chameleon-SAT Performance Comparison with CPU Software:* In a comparative study of solution times for SAT problems, Fig. 6 reveals the significant advantages of the Chameleon-SAT hardware accelerator over traditional CPU-based software solvers. The experimental results show that the hardware accelerator generally achieved 8.39–31.00-times speedup in solution times under various algorithmic operations. Especially in the CTV 2.5 clusters, when the problem size reached the maximum of 120 variables, the hardware accelerator achieved 31-fold acceleration, greatly improving the computational speed and efficiency. In addition, in the CTV 3.5 clusters, the hardware accelerator also performed well, achieving a 12-fold improvement in solution times, further proving its applicability in a wide range of application scenarios. In difficult SAT problem solving, such as CTV 4.3 hard SAT problems, the Chameleon-SAT accelerator was still able to maintain 8.39-times acceleration, demonstrating its stability and reliability in dealing with complex problems. In summary, the Chameleon-SAT hardware accelerator showed excellent performance compared with state-of-the-art SAT accelerators, providing acceleration for versatile SAT problems.

TABLE I
COMPARISON WITH THE STATE-OF-THE-ART SAT ACCELERATORS

	This Work Chameleon-SAT	ASP DAC'21 HW-BCP [28]	Sci. Rep.'23 AIMS [29]	ISSCC'23 Snap-SAT [20]	ISSCC'24 VIP-SAT [21]
Compute Model	ASIC Mixed-signal	CPU+ASIC Digital	ASIC Digital	ASIC Digital	ASIC Digital
Technology	28 nm CMOS	65 nm/7 nm CMOS	45 nm CMOS	65 nm CMOS	65 nm CMOS
In Memory Computation	Y	Y	Y	Y	Y
SRAM Access Rate	Low	Medium	Low	Low	Medium (Read Only)
Adaptability	Y	Y	N	N	N
Algorithm	Complete& Incomplete DPLL/ CDCL/ Local Search	Complete MiniSAT2	Incomplete Ising Machine	Incomplete WalkSAT	Incomplete Stochastic
Test Case	Hard 3-SAT (CTV = 4.30)	N/A N/A	Hard 3-SAT (CTV = 4.30)	Hard 3-SAT (CTV = 4.30)	Hard 3-SAT (CTV = 4.55)
Max. Number of Var	128	N/A	500	128	50
Max. Number of Cla	2048	N/A	1917	1024	218
Solution Time	1.1 μ s (50-var/ 215-cla)	N/A N/A	1.329 μ s (50-var/ 218-cla)	713 μ s (60-var/ 258-cla)	18.7 μ s (50-var/ 218-cla)
Energy	2.849 nJ ^a (50-var/ 215-cla)	N/A N/A	N/A N/A	1098 nJ (60-var/ 258-cla)	20.8 nJ (50-var/ 218-cla)
Area	18 mm^2	4 cm^2 / 3.3 cm^2	169 mm^2	0.93 mm^2	1.116 mm^2

^aEnergy measured for PIM (Processing-in-Memory) array only.

2) *Chameleon-SAT Performance Comparison with ASIC Hardware*: A comparative analysis of Chameleon-SAT and the emerging research on SAT [20], [21], [28], [29] from recent years, as shown in TABLE I. Our SAT hardware accelerator demonstrated performance enhancements on several key performance metrics (solution time and energy consumption) and was the only accelerator capable of supporting all three algorithms simultaneously. A detailed comparison is provided below.

Technology node: We used 28 nm CMOS technology, offering inherent advantages in power and performance compared to 45/ 65 nm counterparts. **In-memory computation**: Chameleon-SAT supported in-memory computation, a key technology for improving computational efficiency, while other devices did not. **Algorithm support**: Our hardware accelerator was the only device that supported DPLL, CDCL, and Ising Machine algorithms, demonstrating the design's strength in algorithmic versatility and flexibility. **Test case**: Focusing on high-CTV 3-SAT problems validates its ability to resolve complex logic constraints. **Scalability**: Chameleon-SAT can handle up to 500 variables and 2048 clauses, the highest among the compared devices, thereby demonstrating its ability to tackle large-scale problems. **Solution time**: Our appliance solved a problem with 50 variables and 218 clauses in only 9.91 μ s, the fastest among the compared devices. **Area**: An area of 18 mm^2 reflects the trade-off for supporting advanced functionalities at scale. In summary, our SAT hardware accelerator demonstrated excellent performance in terms of algorithm support, efficiency, energy efficiency, and scalability.

3) *Chameleon-SAT Quality Function for Adaptive Mode*: To build up the adaptive mode, we conducted various test cases to build up the quality function needs. The results illustrated and presented in Fig. 7 for our quality function with $\alpha = 1$ and $\beta = 0$, show the optimal algorithm choices for different problem sizes and complexities. The check marks indicate the highest-scoring, most-suitable algorithms for specific condi-

tions of size and complexity.

Generally, DPLL emerged as a preferred choice for solving time for smaller problem sizes, such as 50 variables, even with low to medium complexity. However, as the problem size increased beyond 80 variables, local search and CDCL became more appropriate algorithms, particularly under high-complexity conditions. Notably, CDCL excelled and scored significantly higher, demonstrating its suitability for solving large-scale SAT problems with high complexity.

Furthermore, when completeness is required, CDCL can meet this demand while still performing well. Conversely, when completeness is not a requirement, local search becomes a viable option.

V. CONCLUSION

This work presents Chameleon-SAT, a mixed-signal in-memory computing accelerator that enhances energy consumption and solution time performance in Boolean satisfiability solving, offering an adaptive solution to solving SAT problems.

It is the first accelerator to support both complete and incomplete algorithms, efficiently handling a wide range of SAT problem scales and complexities. This results in substantial speed improvements over traditional and emerging solvers, making it a promising solution for diverse data-intensive applications. It achieves small-scale, high-complexity cases ($\geq 90 \times$ for 20 variables/ 86 clauses, satisfiable problems), medium-scale, structured cases ($\geq 19 \times$ for 50 variables/ 215 clauses, unsatisfiable problems) and large-scale, high-complexity cases ($\geq 7 \times$ for 100 variables/ 430 clauses, satisfiable problems).

ACKNOWLEDGMENT

This work is supported in part by the National Key R&D Program of China (Grant No. 2024YFB3108102), and in part by the National Natural Science Foundation of China (Grant No. U22B2024).

REFERENCES

- [1] S. Cook, "The complexity of theorem proving procedures," in *Proc. 3rd Annu. ACM Symp. Theory Comput.*, 1971, pp. 151–158.
- [2] J. Gu, "Global optimization for satisfiability (SAT) problem," *IEEE Trans. Knowl. Data Eng.*, vol. 6, no. 3, pp. 361–381, 1994.
- [3] H. Zhou, R. Jiang, and S. Kong, "CycSAT: SAT-based attack on cyclic logic encryptions," in *Proc. IEEE/ACM Int. Conf. Comput.-Aided Des. (ICCAD)*, 2017.
- [4] M. S. Chowdhury, J. You, C. Chen, and D. Johnson, "Guiding CDCL SAT search via random exploration amid conflict depression," in *Proc. AAAI Conf. Artif. Intell.*, vol. 34, no. 2, 2020, pp. 1428–1435.
- [5] S. Yu, M. Liu, W. Dou, X. Liu, and S. Zhou, "Networking for big data: A survey," *IEEE Commun. Surv. Tutor.*, vol. 19, no. 1, pp. 531–549, 2016.
- [6] H. Kautz, A. Sabharwal, and B. Selman, "Incomplete algorithms," in *Handbook of Satisfiability*. Amsterdam, Netherlands: IOS Press, 2009, pp. 185–203.
- [7] M. Davis and H. Putnam, "A computing procedure for quantification theory," *J. ACM*, vol. 7, no. 3, pp. 201–215, Jul. 1960. [Online]. Available: <https://doi.org/10.1145/321033.321034>
- [8] R. Nieuwenhuis, A. Oliveras, and C. Tinelli, "Abstract DPLL and abstract DPLL modulo theories," in *Proc. Int. Conf. Logic Program. Artif. Intell. Reason. (LPAR)*, 2004, pp. 36–50.
- [9] M. Luo, C.-M. Li, F. Xiao, F. Many, and Z. Lu, "An effective learnt clause minimization approach for CDCL SAT solvers," in *Proc. 26th Int. Joint Conf. Artif. Intell. (IJCAI)*, 2017.
- [10] L. Zhang, C. Madigan, M. Moskewicz, and S. Malik, "Efficient conflict driven learning in a Boolean satisfiability solver," in *Proc. IEEE/ACM Int. Conf. Comput.-Aided Des. (ICCAD)*, 2001, pp. 279–285.
- [11] B. Selman, H. A. Kautz, and B. Cohen, "Local search strategies for satisfiability testing," in *Cliques, Coloring, Satisfiability*, vol. 26, DIMACS, 1993, pp. 521–532.
- [12] D. Kim, N. M. Rahman, and S. Mukhopadhyay, "Presto: A processing-in-memory-based k-SAT solver using recurrent stochastic neural network with unsupervised learning," *IEEE J. Solid-State Circuits*, vol. 59, no. 7, pp. 2310–2320, 2024.
- [13] G. Audemard and L. Simon, "On the glucose SAT solver," *Int. J. Artif. Intell. Tools*, vol. 27, no. 1, pp. 1–25, 2018. [Online]. Available: <https://api.semanticscholar.org/CorpusID:49573977>
- [14] M. Moskewicz, C. Madigan, Y. Zhao, L. Zhang, and S. Malik, "Chaff: Engineering an efficient SAT solver," in *Proc. 38th Des. Autom. Conf.*, 2001, pp. 530–535.
- [15] N. Een and N. Sorensson, "An extensible SAT-solver," in *Theory Appl. Satisfiability Test.*, E. Giunchiglia and A. Tacchella, Eds. Berlin, Germany: Springer, 2004, pp. 502–518.
- [16] M. Fleury and A. Biere, "Mining definitions in Kissat with Kittens," *Form. Methods Syst. Des.*, vol. 60, no. 1, pp. 381–404, 2022. [Online]. Available: <https://doi.org/10.1007/s10703-023-00421-2>
- [17] M. Osama, A. Wijs, and A. Biere, "Certified SAT solving with GPU accelerated inprocessing," *Form. Methods Syst. Des.*, vol. 62, no. 1, pp. 79–118, 2024. [Online]. Available: <https://doi.org/10.1007/s10703-023-00432-z>
- [18] G.-J. Nam, K. A. Sakallah, and R. A. Rutenbar, "Satisfiability-based layout revisited: Detailed routing of complex FPGAs via search-based Boolean SAT," in *Proc. ACM/SIGDA Int. Symp. Field-Program. Gate Arrays (FPGA)*, Monterey, CA, USA, 1999.
- [19] R. Afoakwa, Y. Zhang, U. K. R. Vengalam, Z. Ignjatovic, and M. Huang, "BRIM: Bistable resistively-coupled Ising machine," in *Proc. IEEE Int. Symp. High-Perform. Comput. Archit. (HPCA)*, 2021, pp. 749–760.
- [20] S. Xie et al., "SNAP-SAT: A one-shot energy-performance-aware all-digital compute-in-memory solver for large-scale hard Boolean satisfiability problems," in *Proc. IEEE Int. Solid-State Circuits Conf. (ISSCC)*, Feb. 2023, pp. 420–422.
- [21] C. Shim, J. Bae and B. Kim, "30.3 VIP-Sat: A Boolean Satisfiability Solver Featuring 5×12 Variable In-Memory Processing Elements with 98% Solvability for 50-Variables 218-Clauses 3-SAT Problems," 2024 IEEE International Solid-State Circuits Conference (ISSCC), San Francisco, CA, USA, 2024, pp. 486–488, doi: 10.1109/ISSCC49657.2024.10454397.
- [22] D. Patterson et al., "A case for intelligent RAM," *IEEE Micro*, vol. 17, no. 2, pp. 34–44, 1997.
- [23] M. Oskin, F. Chong, and T. Sherwood, "Active pages: A computation model for intelligent memory," in *Proc. 25th Annu. Int. Symp. Comput. Archit.*, 1998, pp. 192–203.
- [24] J. Draper et al., "The architecture of the DIVA processing-in-memory chip," in *Proc. 16th Int. Conf. Supercomput. (ICS)*, 2002.
- [25] D. Elliott, M. Stumm, W. Snelgrove, C. Cojocaru, and R. McKenzie, "Computational RAM: Implementing processors in memory," *IEEE Des. Test Comput.*, vol. 16, no. 1, pp. 32–41, 1999.
- [26] S. Yin et al., "Monolithically integrated RRAM- and CMOS-based in-memory computing optimizations for efficient deep learning," *IEEE Micro*, vol. 39, no. 6, pp. 54–63, 2019.
- [27] H. Wang et al., "A 32.2 TOPS/W SRAM compute-in-memory macro employing a linear 8-bit C-2C ladder for charge domain computation in 22nm for edge inference," in *Proc. IEEE Symp. VLSI Technol. Circuits*, 2022, pp. 36–37.
- [28] S. Park, J.-W. Nam, and S. K. Gupta, "HW-BCP: A custom hardware accelerator for SAT suitable for single chip implementation for large benchmarks," in *Proc. Asia South Pac. Des. Autom. Conf. (ASP-DAC)*, 2021, pp. 29–34.
- [29] A. Sharma, M. Burns, A. Hahn, and M. Huang, "Augmenting an electronic Ising machine to effectively solve Boolean satisfiability," *Sci. Rep.*, vol. 13, no. 1, p. 22858, 2023.

UniCoS: A Unified Neural and Accelerator Co-Search Framework for CNNs and ViTs

Wei Fu^{1*}, Wenqi Lou^{2*†}, Cheng Tang^{1,2}, Hongbing Wen^{1,2}, Yunji Qin¹, Lei Gong¹, Chao Wang^{1†}, Xuehai Zhou¹

¹ University of Science and Technology of China, China.

² Suzhou Institute of Advanced Research, University of Science and Technology of China, China.
{louwenqi, cswang}@ustc.edu.cn

ABSTRACT

Current algorithm-hardware co-search works often suffer from lengthy training times and inadequate exploration of hardware design spaces, leading to suboptimal performance. This work introduces UniCoS, a unified framework for co-optimizing neural networks and accelerators for CNNs and Vision Transformers (ViTs). By introducing a novel training-free proxy that evaluates accuracy within seconds and a clustering-based algorithm for exploring heterogeneous dataflows, UniCoS efficiently navigates the design spaces of both architectures. Experimental results demonstrate that the solutions generated by UniCoS consistently surpass state-of-the-art (SOTA) methods (e.g., 3.54× energy-delay product (EDP) improvement with a 1.76% higher accuracy on ImageNet) while requiring notably reduced search time (up to 48×, ~3 hours). The code is available at <https://github.com/mine7777/Unicos.git>.

KEYWORDS

zero-shot proxy, heterogeneous multi-core architecture, hardware software co-exploration

1 INTRODUCTION

In recent years, CNNs and ViTs have become dominant backbones in computer vision, driving intense competition in the field [6]. Extensive research explores the design spaces to improve performance [24], yet identifying a definitive superior architecture remains challenging. Simultaneously, these models' high computational and storage demands have driven the development of specialized hardware accelerators, such as Application-Specific Integrated Circuits (ASICs) [4, 8, 22] and Field-Programmable Gate Arrays (FPGAs) [10, 20, 28], optimized for specific models to enhance performance. Recent research highlights the interdependence between hardware optimization and network architectural design, showing that focusing exclusively on either aspect often leads to suboptimal solutions [11, 19].

Consequently, neural-accelerator co-search has been developed to explore better customization and acceleration under predefined accuracy and resource constraints [5, 29]. These approaches typically employ an iterative *two-stage* search strategy to manage the exponential complexity of algorithmic and hardware parameter spaces. In the first stage, candidate networks (also referred to as subnets) are evaluated for accuracy using a one-shot method, which

Table 1: Comparison with Prior Hardware Search or Co-search Work. T&P stands for Tile and Parallism.

Work	Neural Architecture Search			Hardware Dataflow Search		
	CNN	ViT	Train-free	T&P Size	Loop Order	Multicore
GAMMA [12]	–	–	–	✓	✓	✗
HPCA21 [15]	–	–	–	✓	✓	✓
DIAN [29]	✓	✗	✗	✓	✓	✗
DANCE [5]	✓	✗	✗	✓	✓	✗
NAF [21]	✓	✗	✗	✓	✗	✓
Ours	✓	✓	✓	✓	✓	✓

shares operations through a supernet to eliminate the need for separate training of each subnet. In the second stage, the subnets are evaluated against hardware constraints, incorporating feedback on hardware-related losses. This iterative process continues, leveraging accuracy and hardware loss feedback from each epoch to guide the joint optimization.

However, current neural-accelerator co-search methodologies face severe limitations that hinder their optimization potential. *Algorithmic Limitations*: network accuracy evaluation depends on supernet training, which is time-consuming and highly specific to the neural search space. For example, training a supernet for CNNs requires about 100 GPU hours, while ViTs demand up to 576 GPU hours [3]. Moreover, any modification to the search space necessitates retraining or redesigning the supernet. *Hardware Limitations*: The exploration of the hardware design space remains constrained. Existing approaches mainly focus on single-core accelerators with a single dataflow (e.g., tile size and loop order), failing to accommodate the computational heterogeneity across model layers [15]. While developing large accelerators with multiple sub-accelerators or chiplets is gaining traction, it further complicates hardware optimization [2].

To this end, we propose **UniCoS**, a fast and **Unified** framework for the **Co-Search** of neural architectures and accelerators to explore customized CNNs/ViTs and their corresponding ASIC accelerators. Table 1 summarizes the key features of UniCoS and highlights its distinctions from existing approaches. First, we propose a proxy metric on the algorithmic side that eliminates the need for training. This metric utilizes gradient information from the model to rank the accuracy of neural architectures, avoiding the design and training burdens associated with supernets. Second, on the hardware side, we push beyond searching only single-core accelerators by introducing a heterogeneous dataflow design space. Then, we quickly obtain optimized solutions under an expanded design space using layer cluster and heuristic search methods.

DAC '25, June 22–25, 2024, San Francisco, CA, USA
2025. ACM ISBN 978-x-xxxx-xxxx-x/YY/MM...\$15.00
<https://doi.org/10.1145/nnnnnnnn.nnnnnnn>
*Equal contribution. †Corresponding authors.

The major contributions of this paper are as follows:

- We propose a training-free co-exploration framework for accelerators and network architectures, delivering superior application performance. To the best of our knowledge, this framework is the fastest fully open-source co-search solution currently available.
- We develop a zero-cost accuracy evaluation method that assesses the accuracy of a given network within seconds, with adaptability across diverse neural architecture search spaces.
- We integrate a novel heterogeneous dataflow exploration into the framework, utilizing an efficient heuristic-based search method that completes on the desktop-class CPU in just over ten seconds.
- Experimental results show that, for CNN backbones on ImageNet, the network-accelerator solutions generated by UniCoS deliver 3.54× and 12.96× EDP improvements compared to DIAN [29] and DANCE [5] frameworks, respectively. For ViT design space, UniCoS also achieves a favorable accuracy-EDP trade-off.

2 MOTIVATION

Accuracy Assessment Overhead. In one-shot NAS, the supernet serves as an over-parameterized hyper-network, incorporating all candidate operators at each layer. This design results in a linear increase in the size and training overhead of the supernet with operator complexity and quantity. While this approach mitigates the cost of accuracy evaluation, it still demands substantial training data and computational resources (hundreds of GPU hours) [26]. Furthermore, the supernet’s architecture is tightly coupled with the search space, presenting challenges when adapting to varying network design requirements. For instance, modifying the number of candidate operators necessitates retraining the supernet, while altering the types of operators (e.g., introducing transformer operators) may demand a complete redesign of the supernet with a larger dataset for training. Thus, an efficient and plug-and-play method for accuracy assessment is imperative to overcome these limitations.

Hardware Customization Overhead. In existing co-search research, the hardware design space is typically confined to a single dataflow accelerator. However, variations in layer shapes and operators demand different dataflow styles, always leading to software/hardware mismatches on the single accelerator design [15]. To address this, we introduce a heterogeneous dataflow architecture with multiple sub-accelerators in our co-search framework to better balance hardware customization and resource efficiency. While conceptually straightforward, integrating heterogeneous dataflows poses challenges in controlling hardware evaluation costs while exploring an expanded design space. Unlike previous dataflow explorations, co-search requires fast hardware evaluations for numerous subnets, making previous long search times impractical (tens of minutes) [13]. Thus, efficient HW design space exploration is essential for integrating heterogeneous dataflows.

3 CO-SEARCH OVERVIEW

Our proposed co-search framework, shown in Fig. 1, offers a comprehensive solution for designing task-specific neural networks

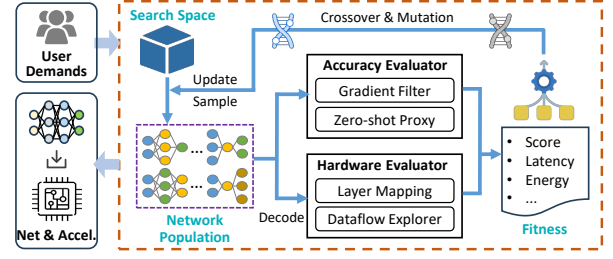


Figure 1: The overview of UniCoS framework.

and accelerators to meet user-defined hardware constraints and dataset specifications.

The exploration process employs an evolutionary algorithm within a well-defined search space incorporating CNN or ViT backbones, where candidate network architectures are represented as individuals. Upon decoding, two specialized evaluators rigorously assess each sampled network for accuracy and hardware performance. In contrast to previous approaches that evaluate network accuracy by inheriting supernet weights, we introduce a training-free *accuracy evaluator* to estimate network performance (Section 4.1). This evaluator computes a zero-shot proxy score derived from gradient information filtered from a mini-batch of training images, ranking networks based on gradient consistency. Notably, this process is completed within mere seconds.

Meanwhile, the *hardware evaluator* assesses the network’s optimal achievable performance under hardware resource constraints (Section 4.2). First, network layers are clustered based on their computational characteristics, and then resources are allocated to these clusters using a greedy strategy, completing the layers to sub-accelerators mapping. Subsequently, heuristic algorithms combined with pruning strategies are employed to efficiently explore the extensive heterogeneous dataflow design space (within tens of seconds). Finally, based on the results from the two evaluators, UniCoS calculates the fitness score for each individual in the network population, defined as
$$\text{Fitness} = \frac{\text{Proxy Score}}{\text{Energy-Delay Product}}$$
. The population’s distribution is then updated through *selection*, *crossover*, and *mutation* processes.

4 PROPOSED METHOD

4.1 Accuracy Proxy Design

Fig. 2 illustrates the workflow of our proposed accuracy evaluator and emphasizes its key distinction from conventional one-shot supernet training methods. Specifically, our approach utilizes proxy metrics to eliminate the substantial computational overhead of training on the entire dataset, a critical advantage of this method. In contrast, the supernet, which is significantly larger than subnets, typically requires extensive training on the entire dataset until convergence. This process demands considerable computational resources and storage capacity from the training platform.

Inspired by [17, 30], we propose a zero-shot proxy for evaluating neural networks, eliminating the design and training overhead associated with supernets at minimal computational cost. We employ a gradient-based evaluation to assess a network’s performance

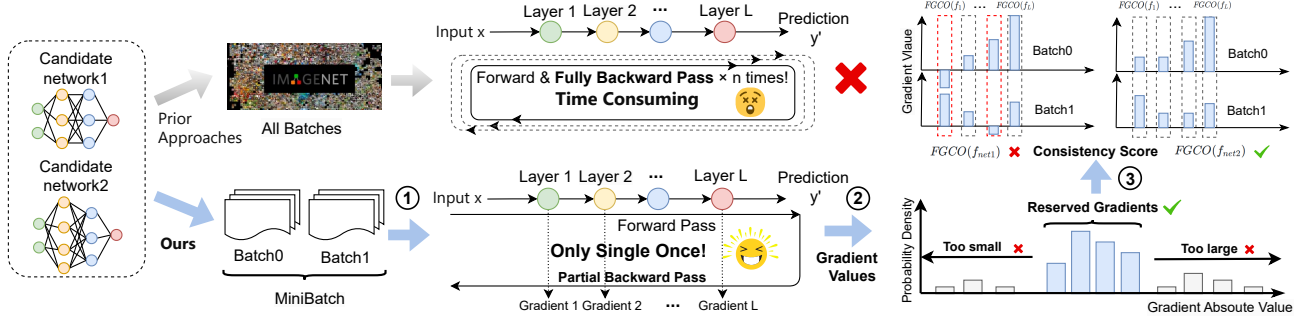


Figure 2: The workflow (①-③) and key features of accuracy evaluator.

potential during training. The network is scored efficiently by extracting gradients from a single forward and backward pass on a mini-batch. This approach significantly reduces the time required for evaluation, as illustrated at the bottom of Fig. 2.

① *Calculate Gradient*. In the first step, we employ Eq. 1 to compute the gradient information for the current batch of data. Let $f(x; \theta)$ denote the candidate network under evaluation, where θ represents the network parameters, and x is the input. Define $Batch = \{(x_1, y_1), \dots, (x_{bs}, y_{bs})\}$ as the set of data points in the training batch, where x_i , y_i , and bs denote the i -th input, the label, and the batch size, respectively. θ_j denote the j -th weight tensor.

$$grad(\theta_j, Batch) = \frac{1}{bs} \sum_{i=1}^{bs} \frac{\partial Loss(y_i, f(x; \theta))}{\partial \theta_j} \quad (1)$$

② *Gradient Filter*: Although gradient information can serve as a valuable indicator of convergence, we argue that indiscriminately leveraging all gradients may introduce evaluation bias, as not all gradients contribute equally to network convergence. We categorize excessively large or small gradients as gradient noise to mitigate this issue, as shown in Fig. 2. To identify the noise range for each network architecture, we adopt a fine-grained gradient preprocessing technique to filter out noisy information before employing the zero-shot proxy for evaluation. Specifically, for each network, all gradients are collected and divided into intervals based on their absolute values. The interval with the high gradient density is selected for proxy computation, while gradients from other intervals are excluded.

③ *Zero-shot Proxy*: Using the Filtered gradients, our proxy computes the final score based on Gradient Consistency, referred to as FGCO. Specifically, we evaluate the consistency of filtered gradients across different batches to capture the oscillation trend during weight updates. Let $Batch_k, k \in [1, n]$ denote one of the mini-batches. The fundamental gradient consistency proxy is then formulated as follows:

$$FGCO(f(x; \theta)) = \sum_{i=1}^L \log \left(\sum_{\theta_j \in \Theta_i} \frac{|E_{Batch_k} fg(\theta_j, Batch_k)|}{\sqrt{Var_{Batch_k} fg(\theta_j, Batch_k)}} \right) \quad (2)$$

Here, L represents the number of network layers, Θ encompasses all network parameters, and Θ_i denotes the parameters of i -th layer. fg means the filter function.

We design the metric based on the consistency of gradients obtained from different batches. Specifically, we calculate this by

dividing the variance of the filtered gradients of each layer by their mean and then accumulating the consistency across all layers. Notably, we retain the sign of the gradients before summing them to capture the oscillation tendencies in gradient directions and weight updates. This metric reflects the expectation that training data from different batches should produce consistent gradients for a single parameter. Higher gradient consistency indicates reduced oscillations in parameter updates, which can lead to faster convergence.

In our experiments, we observed that even using data from just two batches ($n = 2, bs = 32$) is sufficient to achieve strong performance.

4.2 Hardware Dataflow Search

During the hardware evaluation phase, the input is a neural network architecture, and the output includes the hardware performance metrics under its best-fit configuration. As shown in Fig. 3, the hardware exploration process is divided into two stages to navigate the extensive design space efficiently: 1) *Inter-core mapping*: Neural network operators (*i.e.*, layers), the fundamental computational units, are assigned to different computational cores (sub-accelerators). 2) *Intra-core Exploration*: Once operators are allocated, each core's workload is determined, and a sub-evolution algorithm optimizes the dataflow within each core. Below, we will provide a detailed description of each stage.

4.2.1 *Inter-core Mapping*. Neural network computations are abstracted into a sequence of operator executions. These operators are represented using layer parameters: K, C, R, S, X, Y , where K and C denote the number of output and input channels, R and S the kernel dimensions, and X and Y the input feature map dimensions. While convolution operations are directly expressed using these dimensions, matrix multiplications and fully connected layers are modeled as convolutions with a kernel size of 1 ($R \& S = 1$).

Then, operators are clustered in a multi-dimensional space based on their computational parameters, which dictate their behavior on dataflow accelerators. Clustering is performed as follows: 1) When the number of cores is a search parameter, the DBSCAN algorithm [9] determines clusters efficiently in milliseconds. 2) For a given fixed number of cores, the K-means algorithm identifies clusters. This clustering method is adaptable and can be substituted

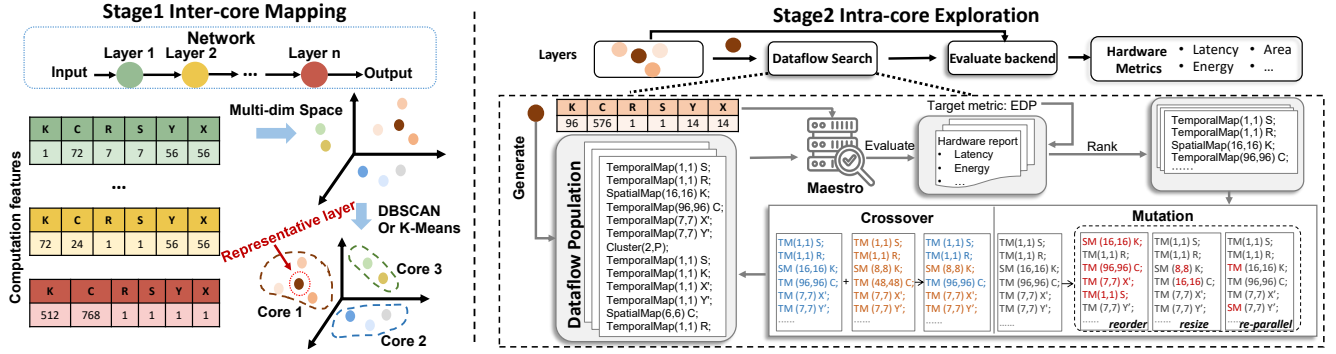


Figure 3: Hardware evaluator workflow with inter-core mapping (left) and intra-core exploration (right).

as needed. Once clustering is complete, hardware resources are allocated proportionally to the computational load of each core.

4.2.2 Intra-core Exploration. With operators clustered and computational resources allocated, the next step involves optimizing the hardware dataflow for each core. Dataflow is characterized by time and space reuse across computational units, described in a data-oriented manner, Maestro [14]. Among that, time mapping (TemporalMap) refers to the scheduling of data, while space mapping (SpatialMap) pertains to the parallelization of computation. By abstracting time and space reuse, the description of a computational core can be simplified to the reuse strategies and the degree of reuse applied across different dimensions.

A genetic algorithm is employed for this exploration, where each dataflow configuration represents an individual. The evaluation backend computes the hardware performance of each individual. During the mutation phase, parameters such as dimension ordering, reuse factors (e.g., tiling and unrolling sizes), and parallelization strategies are adjusted iteratively to enhance performance.

4.2.3 Design Space Pruning. As mentioned earlier, the hardware design space is vast, with exponential growth in complexity. For instance, a single layer with dimensions (1280, 640, 7, 7, 56, 56) results in a search space of approximately $O((7 \times 7 \times 1280 \times 640 \times 56 \times 56 \times 6! \times 6)^2) \approx O(3 \times 10^{29})$, encompassing loop order, tiling size, and parallel dimensions. For multi-core systems with multiple layers, this complexity escalates dramatically. Evaluating a single network may take tens of minutes [13], and the exploration time grows linearly with the number of cores. To accelerate the process, we implement two pruning strategies:

Value-based Pruning. This approach refers to the use of multiples of 4 when considering dataflow reuse dimensions. This approach not only aligns better with the design logic but also effectively reduces the search space.

Strategy-Based Pruning: Instead of evaluating every layer in a cluster, which results in excessively long times, representative layers are selected for each cluster, and the optimal strategy is derived based on these points. This strategy is subsequently extended to the entire cluster, serving as the final optimized solution.

These strategies significantly reduce the search cost, enabling the hardware evaluator to complete evaluations within 15 seconds on a desktop CPU (see Section 5.3), compared to the tens of minutes required in previous studies.

Table 2: Kendall's τ and Spearman's ρ of different proxies on NASBench201 and AutoFormer design space. Img16 means ImageNet16-120.

Benchmark	NASBench-201						Benchmark	AutoFormer	
Dataset	CIFAR10		CIFAR100		ImageNet16		Dataset	ImageNet-1k	
Cor	K	S	K	S	K	S	Cor	K	S
ZenScore	0.29	0.38	0.28	0.36	0.29	0.40	Snip	0.46	0.66
ZiCo	0.61	0.80	0.61	0.81	0.60	0.79	ZiCo	0.62	0.80
GradSign	0.58	0.77	0.59	0.79	0.59	0.78	DSS	0.20	0.30
#Param	0.57	0.75	0.55	0.73	0.52	0.69	Param	0.52	0.72
#FLOP	0.54	0.73	0.51	0.71	0.49	0.67	FLOPs	0.52	0.73
Ours	0.655	0.839	0.658	0.842	0.627	0.814	Ours	0.638	0.820

5 EXPERIMENTS

5.1 Experimental Setup

All experiments were conducted on a desktop-level Intel i5-13400 CPU and an NVIDIA A6000 GPU with 48 GB of video memory. Hardware emulation was performed using MAESTRO [14], a cost simulator validated through RTL simulations for DNN accelerators with flexible dataflows. To evaluate latency, energy, power, and area, the simulator was extended to support multi-sub-accelerator configurations, following methodologies established in prior research.

5.2 Accuracy Evaluator Validation

We ranked networks based on zero-shot proxy scores and evaluated their correlation with true accuracy rankings using Kendall's τ (K) and Spearman's ρ (S), standard correlation coefficients ranging from $[-1, 1]$. Values closer to 1 indicate stronger agreement between the proxy-based and actual accuracy rankings.

Benchmarks: For the CNN design space, we used the NASBench-201 dataset [7], which comprises 15,625 unique architectures with diverse topologies and test accuracy data for CIFAR10, CIFAR100, and ImageNet16-120. In the ViT design space, we randomly sampled 1,000 networks and obtained their accuracies from the Autoformer design space.

Baselines: We compared our proxy against existing zero-shot NAS proxies: ZiCo [17], Zen [18], and GradSign [30] for CNNs, and SNIP [16], ZiCo [17], and DSS [31] for ViTs. Additionally, we included parameter (#Param) and FLOP count (#FLOP) as baselines, as they are critical metrics for neural network evaluation.

Table 3: Test accuracy (%) of searched architectures on NASBench-201 with different zero-shot proxies.

	Optimal Acc.	Naswot	ZiCo	GradSign	#Param	Ours
CIFAR10	94.37	92.96	94.0	93.31	93.80	94.21
CIFAR100	73.51	69.98	71.1	70.33	71.11	72.91
ImageNet16	47.31	44.44	41.8	42.42	41.44	46.34

Table 4: The performance and cost of architecture searched in the AutoF-T design space.

Model	#Param (M)	Top-1 Acc. (%)	Top-5 Acc. (%)	GPU Days
DeiT-T [23]	5.7	72.2	91.1	Manual
PVT-T [25]	13.2	75.1	-	Manual
AutoFormer-Tiny [3]	5.7	74.7	92.6	24
Ours	6.7	75.39	92.85	0.05

Results: Table 2 demonstrates the strong correlation between our proxy scores and model accuracy. Specifically, Kendall’s τ coefficients exceed 0.8 on NASBench-201 and AutoFormer design space, surpassing previous zero-shot methods across all datasets. These results highlight the robustness of our proxy in maintaining high accuracy correlation across diverse search spaces, including CNN and ViT backbones. This performance stems from our effective gradient-based filtering and analysis. As a network evaluation metric, our approach delivers accuracy-related scores within seconds, ensuring both reliability and significant efficiency improvements for collaborative search.

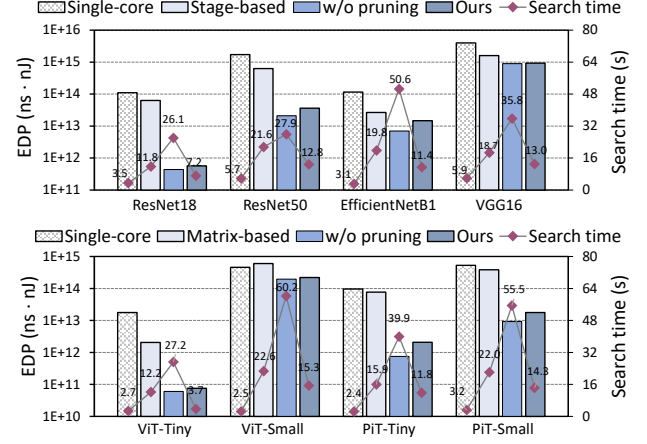
Case Study: We evaluated our accuracy predictor through a NAS experiment on the NASBench-201 and AutoFormer-Tiny (AutoF-T) design spaces. The results, summarized in Tables 3 and 4, highlight the effectiveness of our approach.

For the CNN backbone, our zero-shot proxy identified an architecture achieving test accuracy comparable to the highest reported in the dataset. Notably, it outperformed prior proxy methods, demonstrating its efficacy in discovering optimal architectures. For the ViT backbone, within one hour, our method identified high-accuracy networks that surpassed both existing one-shot NAS methods and manually designed architectures, showcasing its generalizability to diverse search spaces.

5.3 Hardware Evaluator Validation

Baselines: The hardware dataflow search in this framework utilizes a heterogeneous multi-core architecture optimized for search efficiency. We compare our work with single-core-based exploration and typical partition-based exploration. For CNNs, the typical partition strategy is based on resolution stages. For ViTs, the typical partition strategy is based on dimensions of matrix multiplication. We also verify the time reduction brought by our design space pruning.

Results: Fig. 4 presents each approach’s final energy-delay product (EDP) reductions and search times. Leveraging the heterogeneous dataflow architecture, we significantly optimize hardware efficiency, achieving EDP reductions ranging from 4.3 $\times$ to 192.6 $\times$ for CNNs and 2.7 $\times$ to 235.8 $\times$ for ViTs. These substantial gains illustrate the ability of our hardware evaluator to identify optimized designs tailored to the operator’s computation features. Notably,

**Figure 4: Comparison of the hardware search results and the time cost by different search strategies target on CNNs (top) and ViTs (bottom).**

the improvements are more pronounced in smaller networks, where the limited number of layers amplifies the impact of mapping and dataflow strategies on hardware efficiency.

Additionally, our search strategy achieves comparable runtimes to single-core methods while significantly reducing EDPs. This efficiency stems from a design space pruning strategy that, coupled with effective clustering of operator computation features, retains high-quality solutions while eliminating redundant design points. By accelerating the search process by 2.2 $\times$ to 7.4 $\times$ compared to unpruned methods, our approach demonstrates the synergistic impact of clustering and pruning in enhancing search efficiency and hardware optimization.

5.4 Collaborative Search Validation

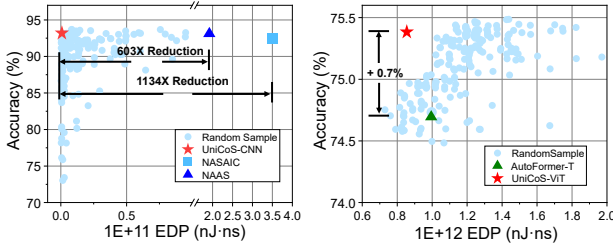
5.4.1 Search Space. We conduct experiments on the co-exploration of neural architecture and accelerator in both the CNN and ViT spaces. In the CNN domain, we used ProxylessNAS [1] as the search space; in the ViT domain, we used Autoformer as the search space. We combined the network results from their respective papers with our hardware search engine to obtain the corresponding EDP values, which served as the baseline.

5.4.2 Collaborative Search Results. We compare our UniCoS framework with several SOTA co-exploration methods. For the CNN search space, comparisons are made with DANCE [5] and DIAN [29]. As shown in Table 5, the network identified by UniCoS achieves superior accuracy under similar search space and hardware resource constraints. When comparing DIAN and DANCE, UniCoS achieves substantially higher gains with 1.76% and 3.81% improvements in accuracy on the Imagenet-1k dataset, respectively, while reducing EDP at significantly higher rates (3.54 $\times$ and 12.96 $\times$). In terms of search efficiency, UniCoS offers a 48 $\times$ speedup over DIAN, reducing search time from 144 hours to just 3 GPU hours, and requires no pretraining.

For ViT search spaces, UniCoS is evaluated against AutoFormer, as no prior co-search frameworks have tackled ViT spaces. Hardware results, obtained using our dedicated evaluator, highlight that UniCoS identifies design points with superior trade-off performance.

Table 5: Comparison of existing co-exploration results on ImageNet-1k dataset.

Method	Space	Acc. (%)	EDP (nJ-ns)	Pretrain	Search Cost ^a
DANCE [5]	CNN	70.15	3.5E+14	Yes	>3h ^b
DIAN [29]	CNN	72.20	9.5E+13	Yes	144h (48×)
Ours	CNN	73.96	2.7E+13	No	3h
AutoF-T [3]	ViT	74.7	9.9E+11	Yes	420h (161×)
Ours	ViT	75.4	8.5E+11	No	2.6h
AutoF-S [3]	ViT	81.7	4.3E+12	Yes	420h (140×)
Ours	ViT	81.5	3.3E+12	No	3.0h
AutoF-B [3]	ViT	82.4	2.5E+13	Yes	420h (131×)
Ours	ViT	82.3	1.6E+13	No	3.2h

^a The reported GPU hours using the baseline's original papers.^b This data does not count the pre-training overhead of the accuracy predictor.**Figure 5: Accuracy and EDP trade-off. CNN design space on the CIFAR10 dataset (left) and ViT design space on the ImageNet-1k dataset.**

Compared to AutoF-B, UniCoS achieves a 36% reduction in EDP and accelerates the search process by 131× (only requiring 3.2 GPU hours), with an accuracy of 82.3% on the ImageNet-1k dataset.

5.4.3 EDP VS. Accuracy. We evaluated our framework's generalization capability across diverse search spaces by sampling 200 networks from the AutoF-T space on ImageNet-1k and the CNN space on CIFAR10, comparing its performance against NAAS [19] and NASAIC [27].

As shown in Fig. 5, the design points (UniCoS-CNN and UniCoS-ViT) identified by our framework are consistently close to the *Pareto front*. In the CNN design space on CIFAR10, our approach achieves a remarkable 603× reduction in EDP compared to NAAS [19]. Likewise, within the Autoformer-Tiny search space, UniCoS demonstrates superior accuracy and EDP, even when using our hardware evaluator for Autoformer, highlighting its effectiveness in jointly optimizing hardware and network architectures.

6 CONCLUSION

This paper presents UniCoS, a unified and efficient framework for co-exploring neural architectures and hardware accelerators within the design spaces of CNNs and ViTs. UniCoS addresses existing limitations by integrating a zero-cost accuracy proxy with a novel heterogeneous dataflow design space optimized through heuristic search. Experimental results demonstrate that UniCoS significantly accelerates joint design space exploration, achieving superior solutions that balance EDP and accuracy compared to prior approaches.

ACKNOWLEDGMENTS

This work was supported in part by the National Key R&D Program of China (Grants 2022YFB4501600, 2022YFB4501603), the National Natural Science Foundation of China (Grant 62172380), Jiangsu Provincial Natural Science Foundation (Grant BK20241818), Youth Innovation Promotion Association CAS (Grant Y2021121), and USTC Research Funds of the Double First-Class Initiative (Grant YD2150002011).

REFERENCES

- [1] Han Cai et al. 2019. ProxylessNAS: Direct Neural Architecture Search on Target Task and Hardware. In *ICLR*.
- [2] Jingwei Cai et al. 2024. Gemini: Mapping and Architecture Co-exploration for Large-scale DNN Chiplet Accelerators. In *HPCA*.
- [3] Minghao Chen et al. 2021. Autoformer: Searching transformers for visual recognition. In *ICCV*.
- [4] Yu-Hsin Chen et al. 2016. Eyeriss: A spatial architecture for energy-efficient dataflow for convolutional neural networks. *ACM SIGARCH computer architecture news* (2016).
- [5] Kanghyun Choi et al. 2021. Dance: Differentiable accelerator/network co-exploration. In *DAC*.
- [6] Xiaohan Ding et al. 2024. UniReplKNet: A Universal Perception Large-Kernel ConvNet for Audio Video Point Cloud Time-Series and Image Recognition. In *CVPR*.
- [7] Xuanyi Dong et al. 2020. NAS-Bench-201: Extending the Scope of Reproducible Neural Architecture Search. In *ICLR*.
- [8] Zidong Du et al. 2015. ShiDianNao: Shifting vision processing closer to the sensor. In *ISCA*.
- [9] Martin Ester et al. 1996. A density-based algorithm for discovering clusters in large spatial databases with noise. In *KDD*.
- [10] Hongxiang Fan et al. 2022. Adaptable butterfly accelerator for attention-based NNs via hardware and algorithm co-design. In *MICRO*.
- [11] Weiwen Jiang et al. 2020. Hardware/software co-exploration of neural architectures. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* (2020).
- [12] Sheng-Chun Kao et al. 2020. GAMMA: Automating the HW mapping of DNN models on accelerators via genetic algorithm. In *ICCAD*.
- [13] Sheng-Chun Kao et al. 2022. Digamma: Domain-aware genetic algorithm for hw-mapping co-optimization for dnn accelerators. In *DATE*.
- [14] Hyoungjun Kwon et al. 2020. Maestro: A data-centric approach to understand reuse, performance, and hardware cost of dnn mappings. *MICRO* (2020).
- [15] Hyoungjun Kwon et al. 2021. Heterogeneous dataflow accelerators for multi-DNN workloads. In *HPCA*.
- [16] Namhoon Lee et al. 2018. SNIP: Single-shot Network Pruning based on Connection Sensitivity. In *ICLR*.
- [17] Guihong Li et al. 2023. ZiCo: Zero-shot NAS via inverse Coefficient of Variation on Gradients. In *ICLR*.
- [18] Ming Lin et al. 2021. Zen-nas: A zero-shot nas for high-performance image recognition. In *ICCV*.
- [19] Yujun Lin et al. 2021. NAAS: Neural accelerator architecture search. In *DAC*.
- [20] Wenqi Lou et al. 2022. OctCNN: A High Throughput FPGA Accelerator for CNNs Using Octave Convolution Algorithm. *IEEE Trans. Comput.* (2022).
- [21] Wenqi Lou et al. 2024. Unleashing Network/Accelerator Co-Exploration Potential on FPGAs: A Deeper Joint Search. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* (2024).
- [22] Liqiang Lu et al. 2021. Sanger: A co-design framework for enabling sparse attention using reconfigurable architecture. In *MICRO-54: 54th Annual IEEE/ACM International Symposium on Microarchitecture*.
- [23] Hugo Touvron et al. 2021. Training data-efficient image transformers & distillation through attention. In *ICML*.
- [24] Ao Wang et al. 2024. Repvit: Revisiting mobile cnn from vit perspective. In *CVPR*.
- [25] Wenhao Wang et al. 2021. Pyramid vision transformer: A versatile backbone for dense prediction without convolutions. In *ICCV*.
- [26] Bichen Wu et al. 2019. Fbnet: Hardware-aware efficient convnet design via differentiable neural architecture search. In *CVPR*.
- [27] Lei Yang et al. 2020. Co-exploration of neural architectures and heterogeneous asic accelerator designs targeting multiple tasks. In *DAC*.
- [28] Shulin Zeng et al. 2024. Flightllm: Efficient large language model inference with a complete mapping flow on fpgas. In *FPGA*.
- [29] Yongan Zhang et al. 2021. DIAN: Differentiable accelerator-network co-search towards maximal dnn efficiency. In *ISLPED*.
- [30] Zhihao Zhang et al. 2022. GradSign: Model Performance Inference with Theoretical Insights. In *ICLR*.
- [31] Qinqin Zhou et al. 2022. Training-free transformer architecture search. In *CVPR*.